

DRL Sessions 6, 7 & 8 — Monte Carlo Methods (Exam Handout)

S6: MC nature, prediction, control, exploration — S7: Off-policy MC, importance sampling, IS ratios —
S8: Race Car example

Session 6 — MC Prediction, Control, and Exploration

1 · Nature of MC Methods and Applications

What is Monte Carlo in RL?

MC methods learn value functions and optimal policies from **complete episodes of experience** — no model of the environment is required. They estimate values by **averaging actual sampled returns**.

Defining characteristics:

1. **Model-free:** no transition function $p(s', r|s, a)$ needed — only sample trajectories.
2. **Episode-based:** must wait until the end of each episode to observe the full return G_t . Therefore MC is restricted to **episodic tasks**.
3. **Law of large numbers:** average return over many episodes converges to true $v_\pi(s)$.
4. **No bootstrapping:** unlike DP and TD, MC does not use other state estimates to update — it uses actual observed returns. This reduces bias but increases variance.

Key contrast with DP:

	Dynamic Programming	Monte Carlo
Requires model?	Yes (full $p(s', r s, a)$)	No (only experience)
Bootstraps?	Yes	No
Task type	Episodic & continuing	Episodic only
Variance	Low	Higher
Bias	Low	None (unbiased estimates)

Applications: board games (Go, Chess — via self-play), simulated environments, blackjack (S&B example), race track control, any task where a model cannot be specified but episodes can be simulated.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

2 · MC Prediction (Policy Evaluation)

Goal: Estimate $v_\pi(s)$ for a given policy π from sampled episodes.

First-Visit vs Every-Visit MC

First-Visit MC: For each episode, only the *first* time state s is visited, record G_t (the return from that time step onward). Average these across episodes.

$$V(s) \leftarrow \text{average of } G_t \text{ for first visits to } s$$

Every-Visit MC: For *every* visit to s in every episode, record G_t . Average all of them.

Both converge to $v_\pi(s)$ as the number of visits $\rightarrow \infty$. First-visit has better theoretical guarantees (each sample is iid); every-visit has lower variance in practice (more samples per episode).

For off-policy MC: always use first-visit (samples from different visits are correlated in off-policy, violating iid assumption).

Incremental update (equivalent, avoids storing all returns):

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)} [G_t - V(S_t)]$$

where $N(S_t)$ = number of times S_t has been visited.

3 · MC Control (Policy Optimisation)

Problem: To improve a policy greedily, we need $q_\pi(s, a)$ (not $v_\pi(s)$), because without a model we cannot compute $\arg \max_a \sum_{s'} p(\dots)$.

MC Control — General Approach

Maintain estimates of $Q(s, a)$ (action-value function) from sampled episodes. Interleave:

1. **Policy Evaluation:** estimate $Q \approx q_\pi$ using MC (average returns after (s, a)).
2. **Policy Improvement:** make π greedy with respect to current Q : $\pi(s) = \arg \max_a Q(s, a)$.

This is GPI applied to MC — converges to π^* if exploration is maintained.

4 · Maintaining Exploration — Why and How

The problem: If the current policy is deterministic (greedy), it never visits many (s, a) pairs. Those pairs never get return estimates. Their $Q(s, a)$ stays at the initial value forever. The policy cannot improve from those states.

Three Solutions to the Exploration Problem

Solution 1 — Exploring Starts (MC-ES):

Every episode starts from a randomly chosen (s_0, a_0) pair, with every pair having > 0 probability of being the start. The target policy can be fully greedy.

Limitation: Only works in simulation (cannot force arbitrary starts in real environments).

Solution 2 — On-Policy ε -Soft / ε -Greedy:

Use a *soft* policy: always assign probability $\geq \varepsilon/|A(s)|$ to every action. The greedy action gets probability $(1 - \varepsilon) + \varepsilon/|A(s)|$.

The policy being evaluated **and** improved is the same ε -soft policy.

Trade-off: Never fully greedy — converges to the best ε -soft policy, not π^* .

Solution 3 — Off-Policy Methods:

Use a **separate behaviour policy** b (explores freely) to generate data, while improving a different **target policy** π (can be greedy).

This is the topic of Session 7.

5 · On-Policy vs. Off-Policy Methods

Key Distinction

On-Policy: The policy used to *generate experience* is the *same* policy being *evaluated and improved*.

Off-Policy: Two separate policies:

- **Behaviour policy** b — generates the episodes (must be exploratory).
- **Target policy** π — the policy being learned (can be deterministic/greedy).

	On-Policy	Off-Policy
Policy count	1 (same for explore & improve)	2 (behaviour b + target π)
Exploration	Forced via ε -soft	b explores freely; π need not
Target policy	ε -optimal (not fully greedy)	Can be fully greedy = π^*
Variance	Lower	Higher (IS ratios multiply variance)
Data reuse	Only current policy's data	Can reuse old/human data
Example	On-policy MC, SARSA	Off-policy MC, Q-Learning

Session 7 — Off-Policy MC & Importance Sampling

6 · Why Off-Policy? — Motivation

Off-policy learning is important because:

- Learn from data generated by a *different* (possibly human) policy.
- Learn *multiple* target policies simultaneously from a single behaviour policy.
- Learn the *optimal greedy* target policy while still exploring (behaviour policy explores).
- Conceptually cleaner: exploration and optimisation are separated.

Coverage requirement: b must *cover* π :

$$\pi(a | s) > 0 \Rightarrow b(a | s) > 0 \quad \forall s, a$$

If π would take an action that b never takes, we can never estimate the value of that action.

7 · Importance Sampling — Definition and Purpose

Problem: Episodes are generated under b , but we want to estimate v_π . The return G_t from an episode under b has the wrong distribution. We need a way to *reweight* these returns.

Importance Sampling (IS)

IS is a statistical technique for estimating the expected value of a function under one distribution (π) using samples from a different distribution (b).

For a trajectory $A_t, S_{t+1}, A_{t+1}, \dots, S_T$ starting from S_t :

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$$

This is the **Importance Sampling Ratio** (IS ratio).

Purpose: It corrects for the mismatch between b and π . An episode where b and π agree on all actions gets ratio ≈ 1 . An episode where b took an action π would rarely take gets ratio ≈ 0 .

Key properties of the IS ratio:

- $\rho_{t:T-1} \geq 0$ always.
- $\mathbb{E}_b[\rho_{t:T-1} G_t] = v_\pi(S_t)$ — so ρG is an unbiased estimate of v_π .
- Variance can be very high if b and π differ greatly.
- If $\pi(A_k | S_k) = 0$ for any step k , then $\rho = 0$ — that episode contributes nothing.

8 · Ordinary vs. Weighted Importance Sampling

Let $\mathcal{T}(s)$ = set of all time steps where state s was visited (first visits). Let $T(t)$ = the terminal time of the episode containing time t .

Two IS Estimators

Ordinary IS:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|}$$

Weighted IS:

$$V(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}}$$

	Ordinary IS	Weighted IS
Bias	Unbiased	Biased (but bias $\rightarrow 0$ with data)
Variance	Unbounded (can be huge)	Bounded; much lower in practice
With 1 episode	Could give extreme estimates	Returns G_t itself (ratio cancels)
Preferred?	Rarely in practice	Yes — almost always preferred

9 · Off-Policy MC Algorithm — Full Details

Off-Policy MC Control (Weighted IS) — How It Works

Setup:

- Target policy π — what we're learning (greedy w.r.t. Q).
- Behaviour policy b — what generates episodes (e.g. ϵ -soft, random).
- $Q(s, a)$ — action-value estimates.
- $C(s, a)$ — cumulative weight denominator.

Per-episode update (backward from T):

Initialise $G \leftarrow 0$, $W \leftarrow 1$. For $t = T - 1, T - 2, \dots, 0$:

$$\begin{aligned}
 G &\leftarrow \gamma G + R_{t+1} \\
 C(S_t, A_t) &\leftarrow C(S_t, A_t) + W \\
 Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)] \\
 \pi(S_t) &\leftarrow \arg \max_a Q(S_t, a) \\
 \text{If } A_t \neq \pi(S_t) : &\textbf{break} \quad (\text{target policy would not take this action}) \\
 W &\leftarrow W \cdot \frac{1}{b(A_t | S_t)} \quad \left(\text{since } \frac{\pi(A_t | S_t)}{b(A_t | S_t)} = \frac{1}{b(A_t | S_t)} \text{ when } \pi \text{ is greedy} \right)
 \end{aligned}$$

Why break when $A_t \neq \pi(S_t)$?

Once an action inconsistent with the target policy is encountered (going backwards), the IS ratio for all earlier steps also becomes 0 (since $\pi(A_t | S_t) = 0$ for non-greedy actions). No further updates are useful for the target policy.

Behaviour vs. Target policy pairing (what works):

Behaviour policy b	Target policy π	Valid?
ϵ -soft (explores all actions)	Greedy π^*	Yes — covers π
Uniform random over all actions	Greedy π^*	Yes — covers everything
Deterministic (same as π)	Greedy π^*	No — no exploration; not coverage
ϵ -soft	Another ϵ -soft	Yes, but usually on-policy is simpler

Session 8 — Race Car Example with MC

10 · The Race Track Problem (S&B Exercise 5.8)

Problem Setup

A race car must navigate a track from a start line to a finish line as quickly as possible.

State s	(x, y, v_x, v_y) — position on track + velocity in both dimensions
Actions	9 choices: increment/decrement/hold each velocity component ($\{-1, 0, +1\}^2$). Velocity components stay in $[0, 4]$ (clamped).
Reward	-1 per step (pushes agent to finish fast).
Terminal	Episode ends when car crosses finish line, or goes off-track (restart).
Constraint	Both velocity components cannot both be 0 (car must always move).

11 · MC-ES Applied to the Race Track

Why MC and not DP?

- State space (x, y, v_x, v_y) is large but manageable.
- More importantly, the *race track geometry* is complex and hard to model analytically — MC learns from simulated runs without needing to write out all transition probabilities.

Algorithm (MC with Exploring Starts):

1. Initialise $Q(s, a)$ arbitrarily, $\pi(s) = \arg \max_a Q(s, a)$.
2. **Generate episode** with exploring start: pick random starting position on start line + random velocity. Follow π (plus random action at $t = 0$).
3. **Backward pass**: for each (S_t, A_t) in the episode (first-visit): Compute G_t (sum of -1 's until finish). Update $Q(S_t, A_t) \leftarrow$ average of observed G 's. Update $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$.
4. Repeat until policy stabilises.

Worked Trace — Simplified 1-D Race Track

Setup: 1-D version. Positions 0 (start) to 6 (finish). Velocity $v \in \{1, 2\}$. State = (position, velocity). Actions: **Acc** (increase v by 1, max 2) or **Hold**. Reward = -1 per step. $\gamma = 1$ (episodic).

Episode 1 (exploring start at $s = (0, 1)$, random first action = **Acc**):

$$(0, 1) \xrightarrow{\text{Acc}} (2, 2) \xrightarrow{\text{Hold}} (4, 2) \xrightarrow{\text{Hold}} (6, 2) \quad G = -3$$

Update $Q(0, 1, \text{Acc}) \leftarrow -3$. $Q(2, 2, \text{Hold}) \leftarrow -2$. $Q(4, 2, \text{Hold}) \leftarrow -1$.

Episode 2 (start at $s = (0, 1)$, random first action = **Hold**):

$$(0, 1) \xrightarrow{\text{Hold}} (1, 1) \xrightarrow{\text{Acc}} (3, 2) \xrightarrow{\text{Hold}} (5, 2) \xrightarrow{\text{Hold}} (7 \rightarrow \text{finish}) \quad G = -4$$

$Q(0, 1, \text{Hold}) \leftarrow -4 < Q(0, 1, \text{Acc}) = -3$.

After Episode 1: $\pi(0, 1) = \text{Acc}$ (return -3 beats -4). The policy correctly learns: *accelerate early to reach finish in fewer steps*.

Key insight from race track:

- MC is *ideal* here: each lap is a complete episode with a clear terminal reward.
- Exploring starts ensure all velocity–position combinations are tried.

- Over many episodes, $Q(s, a)$ converges and the car learns to accelerate on straight sections, decelerate near tight corners.

12 · Summary Comparison — MC Methods

Method	Policy	Exploration	Key property
MC Prediction	Fixed π	N/A	Estimates v_π or q_π
MC-ES	Target only	Exploring starts	Can find π^* ; simulation only
On-Policy MC	ε -soft	ε -greedy	Real-world safe; finds best ε -soft
Off-Policy MC	b (explore) + π (greedy)	Via b	Finds π^* ; high variance

13 · Exam Key-Takeaways — Sessions 6, 7, 8

Must Know — MC Methods

1. **MC is model-free, episode-based.** Learns by averaging returns. Works only for episodic tasks.
2. **First-visit vs every-visit:** both converge; use first-visit for off-policy.
3. **Exploration problem:** greedy MC never tries unexplored (s, a) — use ES, ε -soft, or off-policy.
4. **On-policy:** same policy generates data *and* is improved. Can only find best ε -soft, not π^* .
5. **Off-policy:** behaviour policy b explores; target policy π is improved. Can find π^* .
6. **Coverage:** b must cover π — $\pi(a|s) > 0 \Rightarrow b(a|s) > 0$.
7. **IS ratio:** $\rho_{t:T-1} = \prod_{k=t}^{T-1} \pi(A_k|S_k)/b(A_k|S_k)$. Reweights returns from b to estimate expectation under π .
8. **Ordinary IS:** unbiased, high variance. **Weighted IS:** biased but much lower variance — preferred.
9. **Off-policy MC breaks** when $A_t \neq \pi(S_t)$ (IS ratio becomes 0 for all earlier steps).
10. **Race track:** MC-ES with state = (pos, vel), reward = $-1/\text{step}$, learns fast-line policy.

DRL EC3 Exam Handout — Sessions 6, 7 & 8 (Monte Carlo Methods). Ref: Sutton & Barto Ch. 5.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27